Case Study - Eksploracja danych

Andrii Khomenko

2026-01-31

```r
packages <- c("tidyverse", "naniar", "zoo", "reshape2", "caret", "OneR", "randomForest", "lubridate")
to_install <- packages[!packages %in% installed.packages()[, "Package"]]
if(length(to_install) > 0) install.packages(to_install)

library(tidyverse)
library(naniar)
library(zoo)
library(reshape2)
library(caret)
library(OneR)
library(randomForest)
library(lubridate)

knitr::opts_chunk$set(echo = TRUE)
```

1) Wczytanie danych i przygotowanie podstawowe

```r
# Wczytanie
df = read.csv("financial_regression.csv")
# Typy + sortowanie
df$date <- as.Date(df$date)
df <- df %>% arrange(date)

glimpse(df)
```

```
## Rows: 3,904
## Columns: 47
## $ date              <date> 2010-01-14, 2010-01-15, 2010-01-18, 2010-01-19, 20~
## $ sp500.open        <dbl> 114.49, 114.73, NA, 113.62, 114.28, 113.92, 111.20,~
## $ sp500.high        <dbl> 115.14, 114.84, NA, 115.13, 114.45, 114.27, 111.74,~
## $ sp500.low         <dbl> 114.42, 113.20, NA, 113.59, 112.98, 111.56, 109.09,~
## $ sp500.close       <dbl> 114.93, 113.64, NA, 115.06, 113.89, 111.70, 109.21,~
## $ sp500.volume      <dbl> 115646960, 212252769, NA, 138671890, 216330645, 344~
## $ sp500.high.low    <dbl> 0.72, 1.64, NA, 1.54, 1.47, 2.71, 2.65, 1.00, 1.43,~
## $ nasdaq.open       <dbl> 46.26, 46.46, NA, 45.96, 46.27, 46.06, 45.34, 44.39~
## $ nasdaq.high       <dbl> 46.5200, 46.5500, NA, 46.6400, 46.6040, 46.3500, 45~
## $ nasdaq.low        <dbl> 46.22, 45.65, NA, 45.95, 45.43, 45.30, 44.04, 44.12~
## $ nasdaq.close      <dbl> 46.39, 45.85, NA, 46.59, 45.92, 45.49, 44.16, 44.31~
```

```
## $ nasdaq.volume      <dbl> 75209000, 126849300, NA, 84388200, 145680000, 20493~
## $ nasdaq.high.low    <dbl> 0.3000, 0.9000, NA, 0.6900, 1.1740, 1.0500, 1.4400,~
## $ us_rates_.         <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 0.1~
## $ CPI                <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 217~
## $ usd_chf            <dbl> 1.0206, 1.0264, NA, 1.0340, 1.0453, 1.0426, 1.0400,~
## $ eur_usd            <dbl> 1.4478, 1.4376, NA, 1.4269, 1.4094, 1.4106, 1.4154,~
## $ GDP                <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
## $ silver.open        <dbl> 18.58, 18.54, NA, 18.53, 18.31, 17.82, 17.13, 17.01~
## $ silver.high        <dbl> 18.7000, 18.5400, NA, 18.7900, 18.3100, 17.9100, 17~
## $ silver.low         <dbl> 18.4500, 18.3600, NA, 18.5000, 17.7800, 17.3200, 16~
## $ silver.close       <dbl> 18.6600, 18.4100, NA, 18.7900, 17.8600, 17.3800, 17~
## $ silver.volume      <dbl> 69804, 205106, NA, 130552, 230907, 198067, 184687, ~
## $ silver.high.low    <dbl> 0.2500, 0.1800, NA, 0.2900, 0.5300, 0.5900, 0.2840,~
## $ oil.open           <dbl> 39.30, 38.97, NA, 38.07, 38.23, 38.22, 37.07, 36.50~
## $ oil.high           <dbl> 39.4400, 39.0200, NA, 38.9499, 38.3500, 38.4100, 37~
## $ oil.low            <dbl> 38.8800, 38.2800, NA, 38.0300, 37.8800, 37.0900, 36~
## $ oil.close          <dbl> 39.0600, 38.4000, NA, 38.9300, 37.8900, 37.2400, 36~
## $ oil.volume         <dbl> 8575821, 12615299, NA, 8180157, 11438685, 13716561,~
## $ oil.high.low       <dbl> 0.5600, 0.7400, NA, 0.9199, 0.4700, 1.3200, 0.7700,~
## $ platinum.open      <dbl> 160.13, 160.68, NA, 162.23, 163.51, 161.86, 154.84,~
## $ platinum.high      <dbl> 161.8500, 161.2500, NA, 165.2000, 163.5100, 162.500~
## $ platinum.low       <dbl> 159.7200, 159.5000, NA, 162.1722, 159.0900, 159.120~
## $ platinum.close     <dbl> 161.45, 160.25, NA, 165.12, 163.22, 159.79, 154.88,~
## $ platinum.volume    <dbl> 228500, 189900, NA, 295900, 438200, 247600, 190100,~
## $ platinum.high.low  <dbl> 2.1300, 1.7500, NA, 3.0278, 4.4200, 3.3800, 3.7100,~
## $ palladium.open     <dbl> 43.93, 45.18, NA, 46.01, 46.59, 46.60, 43.70, 43.68~
## $ palladium.high     <dbl> 45.020, 45.760, NA, 47.080, 47.310, 46.980, 44.386,~
## $ palladium.low      <dbl> 43.8600, 44.4000, NA, 45.7000, 45.1700, 45.0700, 42~
## $ palladium.close    <dbl> 44.8400, 45.7600, NA, 46.9400, 47.0500, 45.3000, 43~
## $ palladium.volume   <dbl> 364528, 442210, NA, 629150, 643198, 388457, 343595,~
## $ palladium.high.low <dbl> 1.1600, 1.3600, NA, 1.3800, 2.1400, 1.9100, 1.7860,~
## $ gold.open          <dbl> 111.51, 111.35, NA, 110.95, 109.97, 108.48, 106.93,~
## $ gold.high          <dbl> 112.37, 112.01, NA, 111.75, 110.05, 108.78, 107.68,~
## $ gold.low           <dbl> 110.7900, 110.3800, NA, 110.8300, 108.4600, 106.610~
## $ gold.close         <dbl> 112.030, 110.860, NA, 111.520, 108.940, 107.370, 10~
## $ gold.volume        <dbl> 18305238, 18000724, NA, 10467927, 17534231, 2574783~
```

```r
summary(df$date)
```

```
##         Min.      1st Qu.       Median         Mean      3rd Qu.         Max.
## "2010-01-14" "2013-09-24" "2017-06-03" "2017-06-03" "2021-02-10" "2024-10-23"
```

Dane zostały wczytane z pliku CSV. Zmienna date została przekonwertowana na typ daty, a następnie dane p

2) Kontrola wymiarów i braków danych

```r
# Wymiary
nrow(df)
```

```
## [1] 3904
```

```r
ncol(df)
```

```
## [1] 47
```

```r
# Zmienne, które analizujemy
vars_na = names(df)

# Tabela z liczbą i procentem braków
na_summary <- tibble(
  variable = vars_na,
  missing_count = sapply(df[vars_na], function(x) sum(is.na(x))),
  missing_percent = round(
    sapply(df[vars_na], function(x) mean(is.na(x)) * 100),
    2
  )
)

na_summary
```

```
## # A tibble: 47 x 3
##    variable       missing_count missing_percent
##    <chr>                  <int>           <dbl>
##  1 date                       0            0
##  2 sp500.open               185            4.74
##  3 sp500.high               185            4.74
##  4 sp500.low                185            4.74
##  5 sp500.close              185            4.74
##  6 sp500.volume             185            4.74
##  7 sp500.high.low           185            4.74
##  8 nasdaq.open              185            4.74
##  9 nasdaq.high              185            4.74
## 10 nasdaq.low               185            4.74
## # i 37 more rows
```

```
Na tym etapie sprawdzono liczbę obserwacji, zmiennych oraz skalę braków danych dla wybranych, istotnych
```

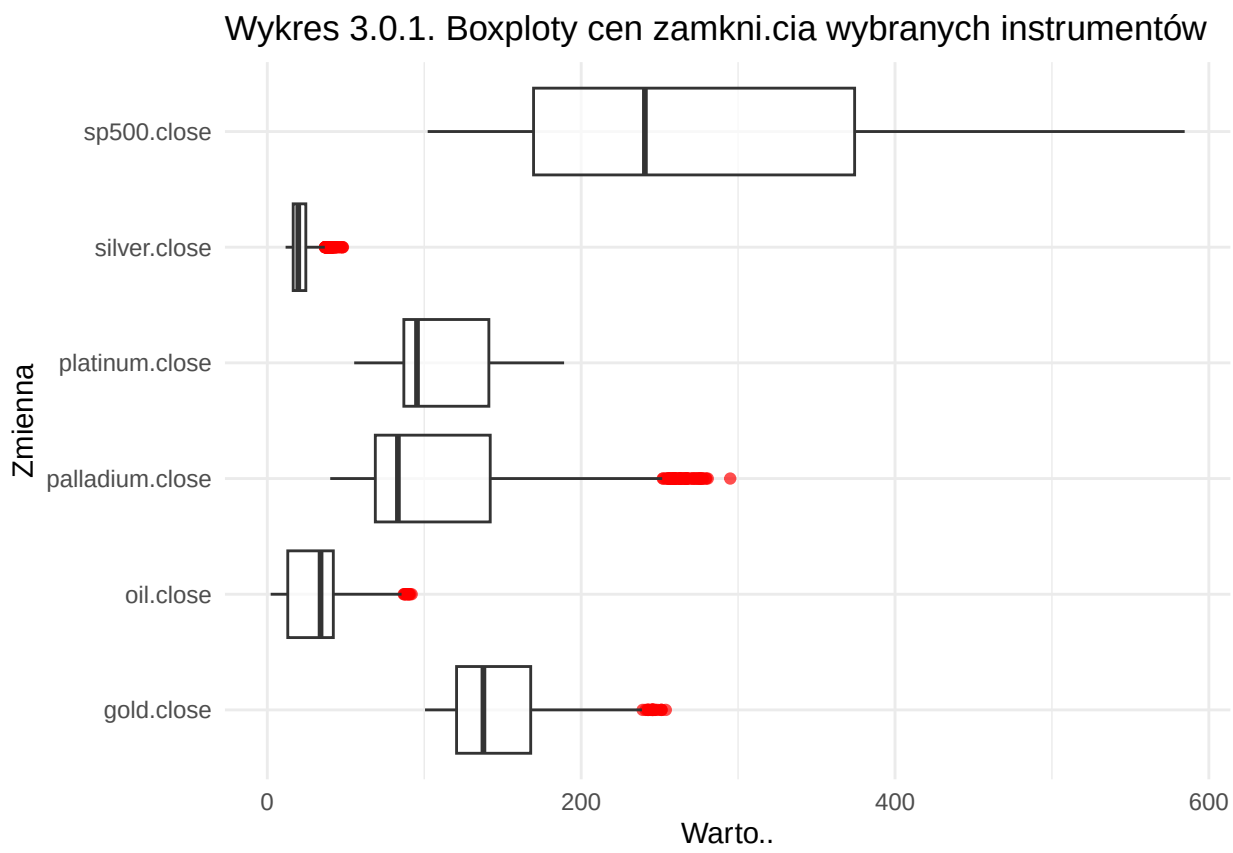
3) Przygotowanie danych (Data Preparation)

3.0.1) Analiza wartości odstających

```r
vars_outliers <- c(
  "gold.close",
  "sp500.close",
  "oil.close",
  "silver.close",
  "platinum.close",
  "palladium.close"
)
```

```r
df_long <- df %>%
  select(all_of(vars_outliers)) %>%
  pivot_longer(
    cols = everything(),
    names_to = "variable",
    values_to = "value"
  )

ggplot(df_long, aes(x = variable, y = value)) +
  geom_boxplot(outlier.colour = "red", alpha = 0.7) +
  coord_flip() +
  theme_minimal() +
  labs(
    title = "Wykres 3.0.1. Boxploty cen zamknięcia wybranych instrumentów",
    x = "Zmienna",
    y = "Wartość"
  )
```

```
## Warning: Removed 1110 rows containing non-finite outside the scale range
## (`stat_boxplot()`).
```



Na wykresie widoczne są wartości odstające, szczególnie dla metali szlachetnych oraz indeksów giełdowych

3.0.2) Outliers w ujęciu czasowym

```
Q1 <- quantile(df$gold.close, 0.25, na.rm=TRUE)
Q3 <- quantile(df$gold.close, 0.75, na.rm=TRUE)
IQR_val <- Q3 - Q1

df <- df %>%
  mutate(
    gold_outlier = gold.close < (Q1 - 1.5 * IQR_val) |
                   gold.close > (Q3 + 1.5 * IQR_val)
  )

ggplot(df, aes(x = date, y = gold.close)) +
  geom_line(alpha = 0.6) +
  geom_point(
    data = df %>% filter(gold_outlier),
    color = "red",
    size = 2
  ) +
  theme_minimal() +
  labs(
    title = "Wykres 3.0.2. Wartości odstające ceny złota w czasie",
    x = "Data",
    y = "Cena zamknięcia złota"
  )
```

Wartości odstające występują głównie w okresach podwyższonej zmienności rynkowej. Nie zostały one usunie

3.0.3) Zmienność (volatility)

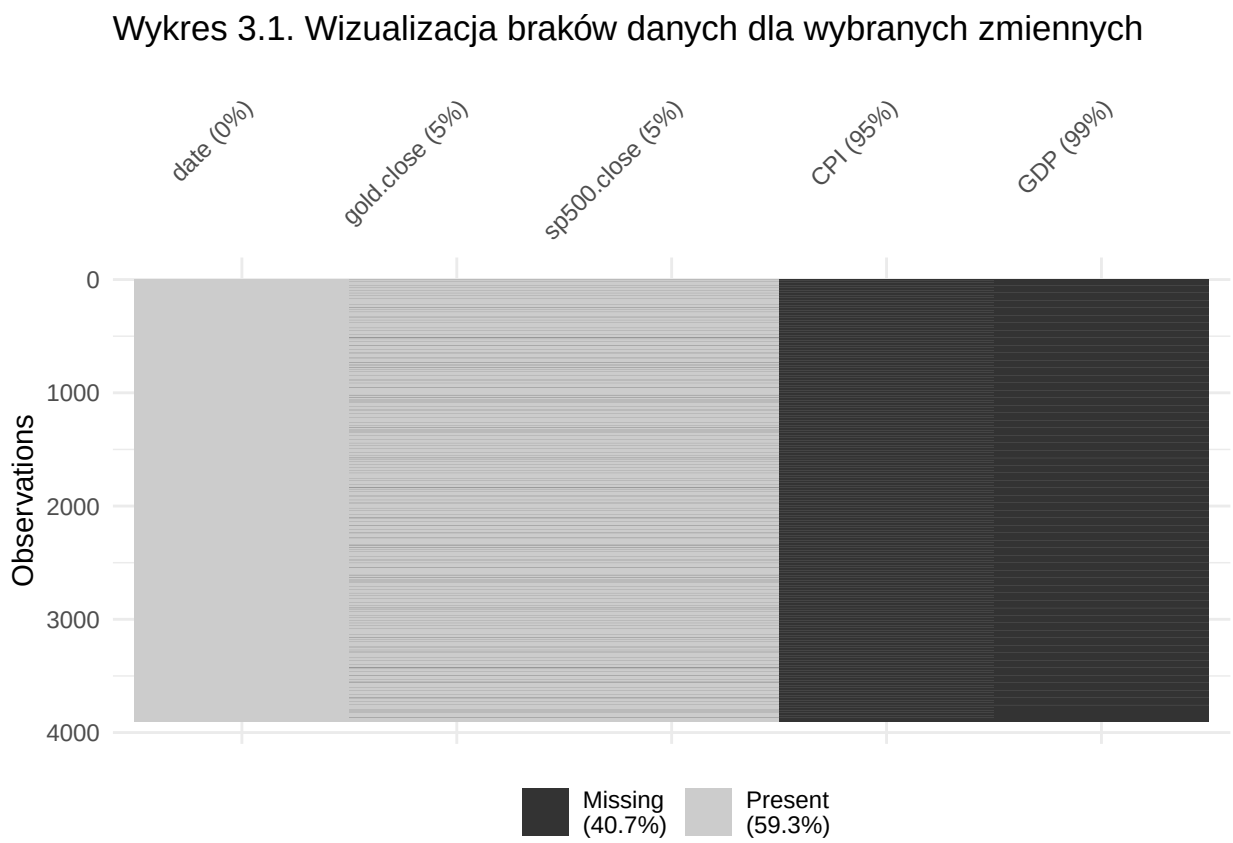
```
df <- df %>%
  mutate(
    gold_return = 100 * (gold.close - lag(gold.close)) / lag(gold.close)
  )

ggplot(df, aes(x = date, y = gold_return)) +
  geom_line(alpha = 0.6) +
  theme_minimal() +
  labs(
    title = "Wykres 3.0.3. Dzienne stopy zwrotu ceny złota",
    x = "Data",
    y = "Stopa zwrotu (%)"
  )
```

```
## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_line()`).
```

Wysoka zmienność dziennych stóp zwrotu potwierdza, że krótkoterminowa predykcja kierunku ceny złota jest

3.1) Missingness plot

```
p_miss <- df %>%
  select(date, gold.close, sp500.close, CPI, GDP) %>%
  vis_miss() +
  labs(title = "Wykres 3.1. Wizualizacja braków danych dla wybranych zmiennych") +
  theme(
    axis.text.x = element_text(angle = 45, hjust = 1)
  )

print(p_miss)
```




```
Na wykresie widać, że zmienne rynkowe mają mniej braków, natomiast CPI i GDP zawierają długie okresy bez
```

3.2) Uzupełnianie danych makroekonomicznych (forward-fill)

```
df <- df %>%
  mutate(
    CPI_original = CPI,
    GDP_original = GDP,
    us_rates_original = us_rates_.
  )
```

```r
df <- df %>%
  mutate(
    CPI = na.locf(CPI, na.rm = FALSE),
    GDP = na.locf(GDP, na.rm = FALSE),
    us_rates_. = na.locf(us_rates_., na.rm = FALSE)
  )

sapply(df[c("CPI", "GDP", "us_rates_.")], function(x) sum(is.na(x)))
```

```
##        CPI        GDP us_rates_.
##         12         55         12
```

```
Zastosowano metodę forward-fill, polegającą na przenoszeniu ostatniej znanej wartości do momentu pojawi
```

```r
df <- df %>%
  filter(
    !is.na(CPI),
    !is.na(GDP),
    !is.na(us_rates_.)
  )

sapply(df[c("CPI", "GDP", "us_rates_.")], function(x) sum(is.na(x)))
```

```
##        CPI        GDP us_rates_.
##          0          0          0
```

```
Początkowe obserwacje bez danych makroekonomicznych zostały usunięte.
```

3.3) Podsumowanie braków danych

```r
num_cols <- df %>%
  select(-date) %>%
  select(where(is.numeric)) %>%
  colnames()

na_summary_all <- tibble(
  variable = num_cols,
  missing_count = sapply(df[num_cols], function(x) sum(is.na(x))),
  missing_percent = round(sapply(df[num_cols], function(x) mean(is.na(x)) * 100), 2)
) %>%
  arrange(desc(missing_count))

na_summary_all %>% filter(missing_count > 0)
```

```
## # A tibble: 47 x 3
##    variable          missing_count missing_percent
##    <chr>                     <int>           <dbl>
##  1 GDP_original               3792           98.5
```

```
##  2 CPI_original              3675          95.5
##  3 us_rates_original         3675          95.5
##  4 gold_return                356           9.25
##  5 usd_chf                    208           5.4
##  6 eur_usd                    208           5.4
##  7 sp500.open                 183           4.75
##  8 sp500.high                 183           4.75
##  9 sp500.low                  183           4.75
## 10 sp500.close                183           4.75
## # i 37 more rows
```

```
df_numeric <- df %>% select(-c(date, GDP_original, CPI_original, us_rates_original))

overall_missing_percent <- round(
  sum(is.na(df_numeric)) / prod(dim(df_numeric)) * 100, 2
)

overall_missing_percent
```

```
## [1] 4.58
```

```
Mamy 4.47% braku danych, więc bezpiecznie możemy usunąć te wiersze, bo po pierwsze to jest mniej niż 5%
```

Usunięcie wierszy z NA

```
df <- df %>% select(-c(GDP_original, CPI_original, us_rates_original)) %>% drop_na()
sum(is.na(df))
```

```
## [1] 0
```

```
nrow(df)
```

```
## [1] 3453
```

3.4) Konstrukcja zmiennej decyzyjnej

```
Tworzymy zmienną klasową:
UP - cena złota wzrosła względem poprzedniego dnia
DOWN - cena złota spadła lub pozostała bez zmian
```

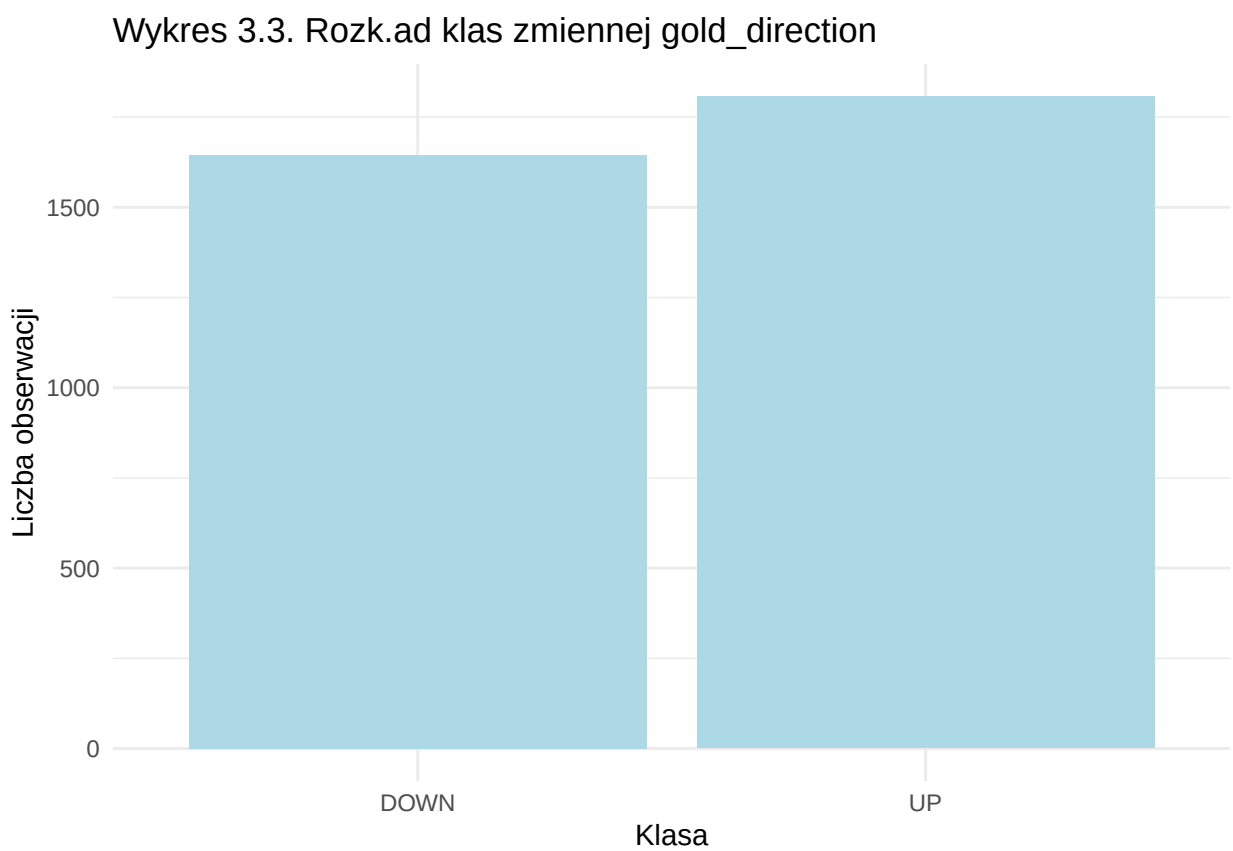
```
df <- df %>%
  arrange(date) %>%
  mutate(gold_diff = gold.close - lag(gold.close))

df <- df %>%
  mutate(gold_direction = ifelse(gold_diff > 0, "UP", "DOWN"))
```

```r
df$gold_direction <- factor(df$gold_direction)

df <- df %>% filter(!is.na(gold_diff))

table(df$gold_direction)
```

```
##
## DOWN   UP
## 1645 1807
```

Rozkład klas zmiennej decyzyjnej

```r
ggplot(df, aes(x = gold_direction)) +
  geom_bar(fill="lightblue") +
  theme_minimal() +
  labs(
    title = "Wykres 3.3. Rozkład klas zmiennej gold_direction",
    x = "Klasa",
    y = "Liczba obserwacji"
  )
```



Rozkład klas jest zbliżony, co jest korzystne z punktu widzenia klasyfikacji.

3.5) Przygotowanie zbioru cech

```
X_numeric <- df %>%
  select(where(is.numeric)) %>%
  select(-gold_diff, -gold.close, -gold_return)

C <- cor(X_numeric)

to_remove <- findCorrelation(C, cutoff = 0.95, names = TRUE)
to_remove
```

```
##  [1] "sp500.high"     "sp500.open"     "sp500.close"    "sp500.low"
##  [5] "GDP"            "nasdaq.high"    "nasdaq.open"    "nasdaq.close"
##  [9] "gold.high"      "gold.open"      "platinum.low"   "platinum.open"
## [13] "platinum.close" "oil.high"       "oil.open"       "oil.close"
## [17] "palladium.high" "palladium.open" "palladium.low"  "silver.high"
## [21] "silver.open"    "silver.close"
```

Finalny zbior cech

```
X_final <- X_numeric %>% select(-all_of(to_remove))

model_df <- df %>%
  select(date, gold_direction) %>%
  bind_cols(X_final)


glimpse(model_df)
```

```
## Rows: 3,452
## Columns: 25
## $ date               <date> 2010-04-06, 2010-04-07, 2010-04-08, 2010-04-09, 20~
## $ gold_direction     <fct> UP, UP, UP, UP, DOWN, DOWN, UP, UP, DOWN, DOWN, UP,~
## $ sp500.volume       <dbl> 110194919, 184432309, 158495871, 132701705, 1102009~
## $ sp500.high.low     <dbl> 0.96, 1.55, 1.37, 0.80, 0.49, 1.04, 1.11, 0.62, 2.5~
## $ nasdaq.low         <dbl> 48.3900, 48.3700, 48.2400, 48.6350, 48.9400, 48.860~
## $ nasdaq.volume      <dbl> 54097100, 77211500, 68314300, 54227300, 44280600, 5~
## $ nasdaq.high.low    <dbl> 0.5100, 0.5000, 0.5800, 0.4150, 0.2600, 0.4900, 0.4~
## $ us_rates_.         <dbl> 0.2, 0.2, 0.2, 0.2, 0.2, 0.2, 0.2, 0.2, 0.2, 0.2, 0~
## $ CPI                <dbl> 217.403, 217.403, 217.403, 217.403, 217.403, 217.40~
## $ usd_chf            <dbl> 1.0704, 1.0716, 1.0723, 1.0672, 1.0598, 1.0573, 1.0~
## $ eur_usd            <dbl> 1.3378, 1.3364, 1.3360, 1.3468, 1.3587, 1.3592, 1.3~
## $ silver.low         <dbl> 17.8800, 18.0400, 17.9400, 18.2400, 18.1100, 17.950~
## $ silver.volume      <dbl> 66513, 208058, 79900, 94081, 102729, 139461, 99485,~
## $ silver.high.low    <dbl> 0.1800, 0.1900, 0.1600, 0.1600, 0.3500, 0.3100, 0.1~
## $ oil.low            <dbl> 41.8300, 41.4500, 40.9000, 40.8100, 40.9000, 40.280~
## $ oil.volume         <dbl> 7307268, 8528509, 6839876, 9041051, 6363110, 105826~
## $ oil.high.low       <dbl> 0.3600, 0.6400, 0.7100, 0.9190, 0.5850, 0.7900, 0.9~
## $ platinum.high      <dbl> 170.6300, 171.8400, 171.4300, 172.4900, 173.7000, 1~
```

```
## $ platinum.volume    <dbl> 69400, 74800, 55800, 53700, 38400, 47200, 70000, 26~
## $ platinum.high.low  <dbl> 0.9699, 1.8400, 1.7999, 1.3000, 1.5200, 1.6000, 0.8~
## $ palladium.close    <dbl> 50.810, 50.890, 50.560, 51.320, 51.940, 52.920, 54.~
## $ palladium.volume   <dbl> 226947, 483783, 382541, 192829, 172620, 365120, 373~
## $ palladium.high.low <dbl> 0.7899, 0.7199, 0.5500, 0.5800, 0.9000, 1.5500, 0.8~
## $ gold.low           <dbl> 110.8300, 111.7700, 112.1300, 112.7400, 112.9800, 1~
## $ gold.volume        <dbl> 10321993, 19127617, 12444414, 16427638, 11118097, 1~
```

```r
C_final <- cor(X_final)
C_long <- melt(C_final)

top_corr <- C_long %>%
  filter(Var1 != Var2) %>%
  mutate(abs_corr = abs(value)) %>%
  rowwise() %>%
  mutate(pair = paste(sort(c(Var1, Var2)), collapse = " | ")) %>%
  ungroup() %>%
  distinct(pair, .keep_all = TRUE) %>%
  arrange(desc(abs_corr)) %>%
  slice(1:10)

top_corr[, c("Var1", "Var2", "value")]
```

```
## # A tibble: 10 x 3
##    Var1            Var2           value
##    <fct>           <fct>          <dbl>
##  1 CPI             nasdaq.low     0.946
##  2 nasdaq.high.low sp500.high.low 0.909
##  3 platinum.high   eur_usd        0.875
##  4 CPI             us_rates_.     0.814
##  5 gold.low        oil.low        0.768
##  6 gold.low        nasdaq.low     0.735
##  7 nasdaq.volume   sp500.volume   0.732
##  8 oil.high.low    oil.low        0.727
##  9 gold.low        CPI            0.708
## 10 nasdaq.high.low nasdaq.low     0.697
```

```
Usunięto cechy o bardzo wysokiej korelacji, aby ograniczyć redundancję informacji.
```

3.6) Podział danych

```r
model_df <- model_df %>% arrange(date)

pred_df <- tail(model_df, 4)
rest_df <- head(model_df, -4)

n_rest <- nrow(rest_df)
split_idx <- floor(0.8 * n_rest)

train_df <- rest_df[1:split_idx, ]
```

```r
test_df  <- rest_df[(split_idx + 1):n_rest, ]

range(train_df$date)
```

```
## [1] "2010-04-06" "2021-11-12"
```

```r
range(test_df$date)
```

```
## [1] "2021-11-15" "2024-10-11"
```

```r
range(pred_df$date)
```

```
## [1] "2024-10-15" "2024-10-18"
```

```r
X_train <- train_df %>% select(-date, -gold_direction)
y_train <- train_df$gold_direction

X_test <- test_df %>% select(-date, -gold_direction)
y_test <- test_df$gold_direction

X_pred <- pred_df %>% select(-date, -gold_direction)
```

```
Podział wykonano z zachowaniem kolejności czasowej. Cztery ostatnie obserwacje wydzielono jako zbiór pr
```

4) Modelowanie

4.1) Model 1 - OneR (One Rule)

```r
oner_model <- OneR(
  gold_direction ~ .,
  data = train_df %>% select(-date)
)
```

```
## Warning in OneR.data.frame(x = data, ties.method = ties.method, verbose =
## verbose, : data contains unused factor levels
```

```r
oner_model
```

```
##
## Call:
## OneR.formula(formula = gold_direction ~ ., data = train_df %>%
##     select(-date))
##
## Rules:
## If platinum.high.low = (0.231,1.57] then gold_direction = UP
## If platinum.high.low = (1.57,2.91]  then gold_direction = DOWN
## If platinum.high.low = (2.91,4.24]  then gold_direction = DOWN
```

```
## If platinum.high.low = (4.24,5.57]  then gold_direction = DOWN
## If platinum.high.low = (5.57,6.91]  then gold_direction = DOWN
##
## Accuracy:
## 1482 of 2758 instances classified correctly (53.73%)
```

```r
oner_pred_test <- predict(oner_model, X_test)

confusionMatrix(
  factor(oner_pred_test, levels = levels(y_test)),
  y_test,
  positive = "UP"
)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction DOWN  UP
##       DOWN  103 117
##       UP    232 238
##
##                Accuracy : 0.4942
##                  95% CI : (0.4563, 0.5322)
##     No Information Rate : 0.5145
##     P-Value [Acc > NIR] : 0.8653
##
##                   Kappa : -0.0223
##
##  Mcnemar's Test P-Value : 1.046e-09
##
##             Sensitivity : 0.6704
##             Specificity : 0.3075
##          Pos Pred Value : 0.5064
##          Neg Pred Value : 0.4682
##              Prevalence : 0.5145
##          Detection Rate : 0.3449
##    Detection Prevalence : 0.6812
##       Balanced Accuracy : 0.4889
##
##        'Positive' Class : UP
##
```

```r
oner_pred_pred <- predict(oner_model, X_pred)

pred_df %>%
  select(date, gold_direction) %>%
  mutate(OneR_prediction = oner_pred_pred)
```

```
##            date gold_direction OneR_prediction
## 3449 2024-10-15             UP              UP
## 3450 2024-10-16             UP              UP
## 3451 2024-10-17             UP            DOWN
## 3452 2024-10-18             UP              UP
```

4.2) Model 2 - Random Forest

```r
ntree_vals <- c(50, 100, 200, 300, 500, 1000, 1500)

oob_error <- sapply(ntree_vals, function(n) {
     rf <- randomForest(
     x = X_train,
     y = y_train,
     ntree = n
   )
   rf$err.rate[n, "OOB"]
})

data.frame(
   ntree = ntree_vals,
   OOB_error = oob_error
)
```

```
##   ntree OOB_error
## 1    50 0.4905729
## 2   100 0.4702683
## 3   200 0.4731690
## 4   300 0.4778825
## 5   500 0.4749819
## 6  1000 0.4717186
## 7  1500 0.4677302
```

```r
set.seed(123)

rf_model <- randomForest(
  x = X_train,
  y = y_train,
  ntree = 1000,
  importance = TRUE
)

rf_model
```

```
##
## Call:
##  randomForest(x = X_train, y = y_train, ntree = 1000, importance = TRUE)
##                Type of random forest: classification
##                      Number of trees: 1000
## No. of variables tried at each split: 4
##
##         OOB estimate of  error rate: 46.27%
## Confusion matrix:
##      DOWN  UP class.error
## DOWN  581 729   0.5564885
## UP    547 901   0.3777624
```

```r
rf_pred_test <- predict(rf_model, X_test)

confusionMatrix(
  factor(rf_pred_test, levels = levels(y_test)),
  y_test,
  positive = "UP"
)
```

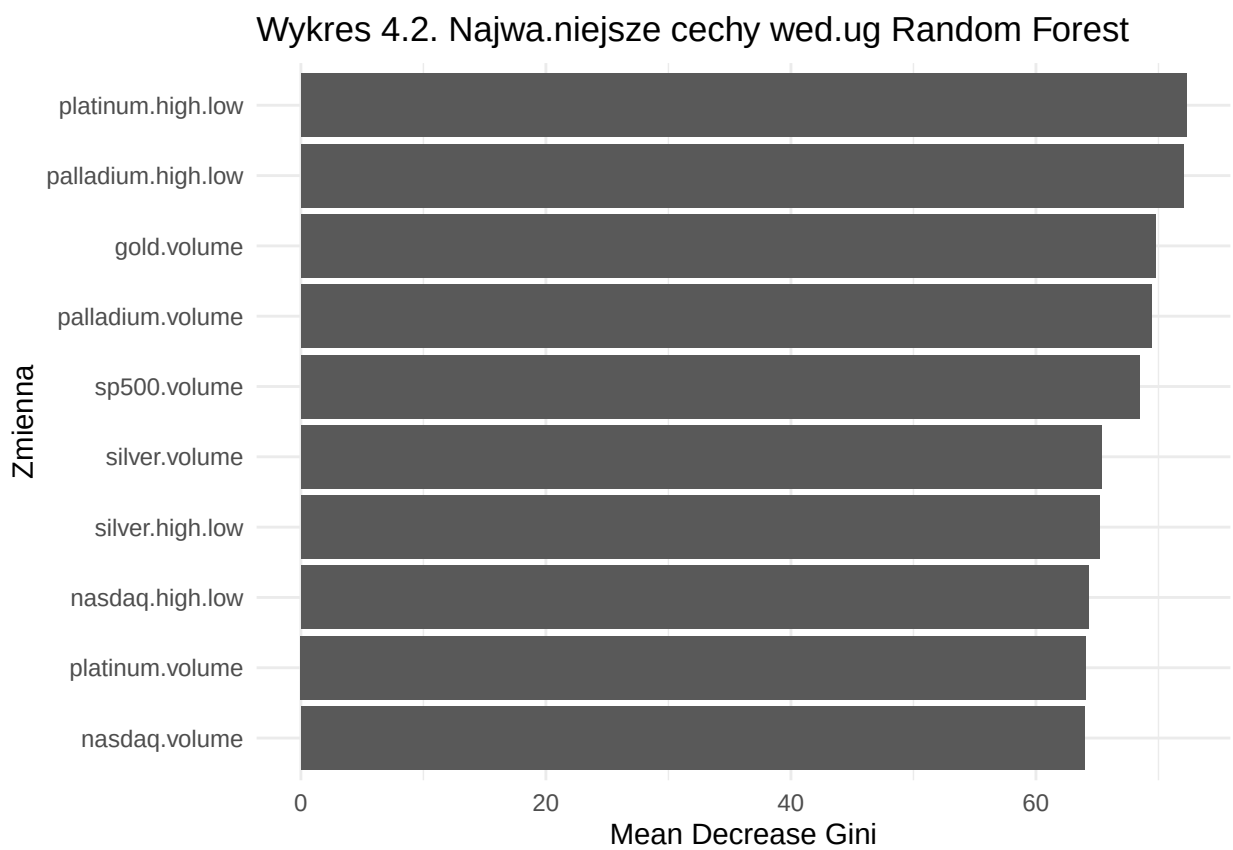
```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction DOWN  UP
##       DOWN   26  31
##       UP    309 324
##
##                Accuracy : 0.5072
##                  95% CI : (0.4693, 0.5452)
##     No Information Rate : 0.5145
##     P-Value [Acc > NIR] : 0.6625
##
##                   Kappa : -0.0099
##
##  Mcnemar's Test P-Value : <2e-16
##
##             Sensitivity : 0.91268
##             Specificity : 0.07761
##          Pos Pred Value : 0.51185
##          Neg Pred Value : 0.45614
##              Prevalence : 0.51449
##          Detection Rate : 0.46957
##    Detection Prevalence : 0.91739
##       Balanced Accuracy : 0.49514
##
##        'Positive' Class : UP
##
```

```r
importance_rf <- importance(rf_model)

importance_rf_df <- data.frame(
  variable = rownames(importance_rf),
  MeanDecreaseGini = importance_rf[, "MeanDecreaseGini"]
) %>%
  arrange(desc(MeanDecreaseGini))

head(importance_rf_df, 10)
```

```
##                              variable MeanDecreaseGini
## platinum.high.low   platinum.high.low         72.25740
## palladium.high.low palladium.high.low         72.04902
## gold.volume               gold.volume         69.76981
## palladium.volume     palladium.volume         69.44390
## sp500.volume             sp500.volume         68.44761
## silver.volume           silver.volume         65.32936
```

```
## silver.high.low       silver.high.low         65.18605
## nasdaq.high.low       nasdaq.high.low         64.28530
## platinum.volume       platinum.volume         64.08072
## nasdaq.volume           nasdaq.volume         63.96333
```

```
ggplot(importance_rf_df[1:10, ],
       aes(x = reorder(variable, MeanDecreaseGini),
           y = MeanDecreaseGini)) +
  geom_col() +
  coord_flip() +
  theme_minimal() +
  labs(
    title = "Wykres 4.2. Najważniejsze cechy według Random Forest",
    x = "Zmienna",
    y = "Mean Decrease Gini"
  )
```




```
rf_pred_pred <- predict(rf_model, X_pred)

pred_df %>%
  select(date, gold_direction) %>%
  mutate(RandomForest_prediction = rf_pred_pred)
```

```
##            date gold_direction RandomForest_prediction
## 3449 2024-10-15             UP                      UP
```

```
## 3450 2024-10-16             UP                      UP
## 3451 2024-10-17             UP                      UP
## 3452 2024-10-18             UP                      UP
```

```
Model Random Forest został wykorzystany jako bardziej złożony algorytm klasyfikacyjny, umożliwiający wyk
```

5) Ewaluacja modeli

```r
oner_cm <- confusionMatrix(
  factor(oner_pred_test, levels = levels(y_test)),
  y_test,
  positive = "UP"
)

rf_cm <- confusionMatrix(
  factor(rf_pred_test, levels = levels(y_test)),
  y_test,
  positive = "UP"
)

evaluation_table <- data.frame(
  Model = c("OneR", "Random Forest"),
  Accuracy = c(oner_cm$overall["Accuracy"], rf_cm$overall["Accuracy"]),
  Precision = c(oner_cm$byClass["Precision"], rf_cm$byClass["Precision"]),
  Recall = c(oner_cm$byClass["Recall"], rf_cm$byClass["Recall"]),
  F1 = c(oner_cm$byClass["F1"], rf_cm$byClass["F1"])
)

evaluation_table
```

```
##           Model  Accuracy Precision    Recall        F1
## 1          OneR 0.4942029 0.5063830 0.6704225 0.5769697
## 2 Random Forest 0.5072464 0.5118483 0.9126761 0.6558704
```